

下一步

使用 Kimi 视觉模型

📄 复制页面



Kimi 视觉模型（包括 `moonshot-v1-8k-vision-preview` / `moonshot-v1-32k-vision-preview` / `moonshot-v1-128k-vision-preview` / `kimi-k2.5` / `kimi-k2.6`）能够理解视觉内容，包括图片文字、图片颜色和物体形状等内容。而最新的 `kimi-k2.6` 模型还能理解视频内容。

使用base64直接上传图片

我们通过以下代码来向 Kimi 提问有关图片的内容：

```
import base64
```

KIMI 开放平台



```
from openai import OpenAI
```

```
client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url="https://api.moonshot.cn/v1",
)
```

```
# 在这里，你需要将 kimi.png 文件替换为你想让 Kimi 识别的图片的地址
image_path = "kimi.png"
```

```
with open(image_path, "rb") as f:
    image_data = f.read()
```

```
# 我们使用标准库 base64.b64encode 函数将图片编码成 base64 格式的 image_url
image_url = f"data:image/{os.path.splitext(image_path)[1].lstrip('.')};base64,{base64.b64encode(open(image_path, 'rb').read()).decode()}"
```

```
completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {"role": "system", "content": "你是 Kimi。"},
        {
            "role": "user",
            # 注意这里, content 由原来的 str 类型变更为一个 list, 这个 list 中包含多个部分
            # 文字 (text) 是一个部分 (part)
            "content": [
                {
                    "type": "image_url", # <-- 使用 image_url 类型来上传图片, 内容为使
                    "image_url": {
                        "url": image_url,
                    },
                },
                {
                    "type": "text",
                    "text": "请描述图片的内容。", # <-- 使用 text 类型来提供文字指令, 例如
                },
            ],
        },
    ],
)
```

KIMI 开放平台



```
print(completion.choices[0].message.content)
```

>

注意，在使用 Vision 模型时，`message.content` 字段的类型由 `string` 变更为 `array[object]`（即 JSON 数组）。额外的，请不要将 JSON 数组序列化后以 `string` 的格式放入 `message.content` 中，这样会导致 Kimi 无法正确识别图片类型，并可能引发 `Your request exceeded model token limit` 错误。

正确的格式：

KIMI

开放平台



```
"model": "kimi-k2.6",
```

```
messages:
```

```
[
```

```
{
```

```
  "role": "system",
```

```
  "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅长中文和英文的"
```

```
},
```

```
{
```

```
  "role": "user",
```

```
  "content":
```

```
  [
```

```
    {
```

```
      "type": "image_url",
```

```
      "image_url":
```

```
      {
```

```
        "url": "data:image/png;base64,..."
```

```
      }
```

```
    },
```

```
    {
```

```
      "type": "text",
```

```
      "text": "请描述这个图片"
```

```
    }
```

```
  ]
```

```
}
```

```
],
```

```
"temperature": 0.3
```

```
}
```

错误的格式：

KIMI

开放平台



```
"model": "kimi-k2.6",
"messages": [
  {
    "role": "system",
    "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅长中文和英文的"
  },
  {
    "role": "user",
    "content": "[{\"type\": \"image_url\", \"image_url\": {\"url\": \"data:"
  }
],
"temperature": 0.3
}
```

使用已上传的图片或视频

上一节的例子中，我们的 `image_url` 为base64编码后的图片。由于视频文件往往更大，所以我们还提供额外方法，您可以先上传图片或视频到 Moonshot，然后通过文件 ID 引用。关于图片或视频的上传，请参阅 [图片理解上传](#)

```
from pathlib import Path
```

KIMI 开放平台



```
from openai import OpenAI
```

```
>
client = OpenAI(
    api_key=os.environ.get("MOONSHOT_API_KEY"),
    base_url="https://api.moonshot.cn/v1",
)
```

在这里，你需要将 video.mp4 文件替换为你想让 Kimi 识别的图片或视频的地址

```
video_path = "video.mp4"
```

```
file_object = client.files.create(file=Path(video_path), purpose="video") # 上传图
```

```
completion = client.chat.completions.create(
    model="kimi-k2.6",
    messages=[
        {
            "role": "system",
            "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你更擅长中文和英文的",
        },
        {
            "role": "user",
            "content": [
                {
                    "type": "video_url",
                    "video_url": {
                        "url": f"ms://{file_object.id}" # 注意这里为 ms:// 而不是 b
                    }
                },
                {
                    "type": "text",
                    "text": "请描述这个视频"
                }
            ]
        }
    ]
)
```

```
print(completion.choices[0].message.content)
```

KIMI 开放平台



注意上面例子中 `video_url.url` 的格式为 `ms://<file-id>`，ms为moonshot storage的缩写，这是 Moonshot 内部引用文件的协议。

支持的格式

图片支持以下格式

- png
- jpeg
- webp
- gif

视频支持以下格式

- mp4
- mpeg
- mov
- avi
- x-flv
- mpg
- webm
- wmv
- 3gpp

Tokens 计算及费用

请求的token消耗。

KIMI 开放平台

一般说来，图片分辨率越高，消耗的token越多；视频由若干张关键帧组成，关键帧的数量越多，分辨率越高，则token消耗越多。

Vision 模型在计费方式上与 **moonshot-v1** 系列模型保持一致，根据模型推理的总 Tokens 计费，详情请查看：

关于token价格，详见 [模型推理价格说明](#)

最佳实践

分辨率

我们推荐图片分辨率不超过4k (4096*2160)，视频分辨率不超过2k (2048*1080)，再高的分辨率只会增加处理时间，也不会对模型理解的效果有提升。

上传文件还是base64

由于我们对请求体的整体大小有限制，所以对于非常大的视频，必须使用上传文件的方式使用视觉理解功能。

对于需要多次引用的图片或视频，我们推荐使用文件上传的方式使用视觉理解功能。

关于上传文件的限制，请参阅 [文件上传](#) 文档。

功能说明及限制

Vision 视觉模型支持的特性包括：

- 多轮对话
- 流式输出
- 工具调用
- JSON Mode

以下功能暂未支持或部分支持

KIMI

开放平台



- URL 格式的图片：不支持，目前仅支持使用 base64 编码的图片内容

>

其他限制：

- 图片数量：Vision 模型没有图片数量限制，但请确保请求的 Body 大小不超过 100M

参数变动说明

在 [chat](#) 文档中有一系列参数，但对于 k2.6/k2.5 系列模型，其行为会有所不同

我们建议用户不要手动设置这些字段，而是使用默认值

参数变动列举如下

字段	是否必须	说明	类型	取值
max_tokens	optional	聊天完成时生成的最大 token 数。	int	默认值为32k，即32768
thinking	optional	新增 该参数控制模型是否启用思考。	object	默认值为 <code>{"type": "enabled"}</code> 。只 <code>{"type":</code> <code>"enabled"}</code> 或 <code>{"type":</code> <code>"disabled"}</code>
temperature	optional	使用什么采样温度。	float	k2.6/k2.5 系列模型使用确定值 1.0, 非模式下将使用默认 0.6。若指定其他值将会报错。
top_p	optional	采样方法。	float	k2.6/k2.5 系列模型使用确定值 0.95。指定其他值，将会报错。

n	optional	为每条输入消息生成多少个结果。	int	k2.6/k2.5 系列模型使用确定值 1。若指定其他值，将会报错。
 开放平台				
presence_penalty	optional	存在惩罚。	float	k2.6/k2.5 系列模型使用固定值 0.0。指定其他值，将会报错。
frequency_penalty	optional	频率惩罚。	float	k2.6/k2.5 系列模型使用确定值 0.0。指定其他值，将会报错。

高级用法

在 Kimi Cli 中使用视觉模型

Kimi Cli 是 Moonshot 推出的开源终端 AI Agent. 随着 k2.6 模型上线，Kimi Cli的能力也有了相应的提升，用户可以使用 Kimi Agent SDK，在自己的程序中调用 Kimi Cli，更方便地使用 Kimi Cli。

以下是一个使用 Kimi Agent SDK 实现的基于动画截图寻找出处的工具 [anime-recognizer](#)

此页面对您有帮助吗？

 是

 否

[多轮对话指南](#)

